ASSIGNMENT - 4.2

2303A51091
Batch : 29

Task Description-1
• Zero-shot: Prompt AI with only the instruction. Write a Python function to determine whether a given number is prime
Expected Output-1
• A basic Python function to check if a number is prime, demonstrating correct logical conditions without relying on examples or additional context

task1.py > ...

```python
def is_prime(n):
    if n <= 1:
        return False

    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False

    return True


# Take input from user
num = int(input("Enter a number: "))

if is_prime(num):
    print(num, "is a Prime number")
else:
    print(num, "is NOT a Prime number")
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python

/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-4.2.py/task1.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-4.2.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-4.2.py/task1.py"
Enter a number: 5
5 is a Prime number
arepallyramcharan@Arepallys-MacBook-Air assignment-4.2.py %
```

Observation:

1. **AI generates a logically correct function based only on instruction.**
2. **Output is usually simple and straightforward.**
3. **May lack advanced optimizations unless explicitly requested.**
4. **Demonstrates that zero-shot works well for common problems.**

Task Description-2

• One-shot: Provide one example: Input: [1, 2, 3, 4], Output: 10 to help AI generate a function that calculates the sum of elements in a list.

Expected Output-2

• A correct conversion function guided by the single example.

```python
def sum_list(numbers):
    total = 0
    for num in numbers:
        total += num
    return total


# Taking input from user
numbers = list(map(int, input("Enter numbers separated by space: ").split()))

print("Sum of elements:", sum_list(numbers))
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-4.2.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-4.2.py/task2.py"
Enter numbers separated by space: 1 2 3 4
Sum of elements: 10
arepallyramcharan@Arepallys-MacBook-Air assignment-4.2.py %
```

Observation:

1. **Modular Design** – One function handles multiple shapes, improving reusability. The single example helps AI clearly understand expected behavior.

2. Output format and logic align closely with the example.

3. Reduces ambiguity compared to zero-shot.

4. Produces more accurate and structured output.

Task Description-3

• Few-shot: Give 2–3 examples to create a function that extracts digits from an alphanumeric string.

Expected Output-3

• Accurate function that returns only the digits from alphanumeric string.

task3.py > ...

```python
def extract_digits(text):
    digits = ""
    for char in text:
        if char.isdigit():
            digits += char
    return digits


# Taking input
text = input("Enter alphanumeric string: ")

print("Digits extracted:", extract_digits(text))
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python

/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-4.2.py/task3.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-4.2.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-4.2.py/task3.py"
Enter alphanumeric string: abnsgt56
Digits extracted: 56
arepallyramcharan@Arepallys-MacBook-Air assignment-4.2.py %
```

Observation:

1. Multiple examples improve pattern recognition.
2. AI correctly understands transformation logic.
3. Fewer logical errors compared to zero-shot.
4. Better handling of edge cases due to examples.

Task Description-4

• Compare zero-shot vs few-shot prompting for generating a function that counts the number of vowels in a string.

Expected Output-4

• Output comparison + student explanation on how examples helped the model.

task4.py > ...

```python
def count_vowels(text):
    vowels = "aeiou"
    count = 0

    for char in text.lower():
        if char in vowels:
            count += 1

    return count


# Taking input
text = input("Enter a string: ")

print("Number of vowels:", count_vowels(text))
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-4.2.py/task4.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-4.2.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-4.2.py/task4.py"
Enter a string: abcdefghijk
Number of vowels: 3
arepallyramcharan@Arepallys-MacBook-Air assignment-4.2.py %
```

.

Observation:

- Zero-shot gives general implementation.
- Few-shot provides more consistent and accurate output.
- Examples guide formatting and logic precision.
- Few-shot reduces misunderstanding of requirements.

Task Description-5

• Use few-shot prompting with 3 sample inputs to generate a function that determines the minimum of three numbers without using the built-in min() function.

Expected Output-5

• A function that handles all cases with correct logic based on example patterns.

task5.py > ...

```python
def minimum_of_three(a, b, c):
    smallest = a

    if b < smallest:
        smallest = b
    if c < smallest:
        smallest = c

    return smallest


# Taking input
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

print("Minimum number is:", minimum_of_three(a, b, c))
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python

/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-4.2.py/task5.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-4.2.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-4.2.py/task5.py"
Enter first number: 12
Enter second number: 23
Enter third number: 34
Minimum number is: 12
arepallyramcharan@Arepallys-MacBook-Air assignment-4.2.py %
```

Observation:

- Examples help AI infer comparison logic correctly.
- Function handles different number order cases properly.
- More reliable than zero-shot for conditional problems.
- Demonstrates that examples improve logical consistency.